C200 Programming Assignment № 3

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

September 27, 2024

Due Date: Friday, October 4, 2024 at 11:00 PM. Please start this homework in a day or so. Do not rush solving–it makes for unpleasant experience and interferes with learning. Please submit your work to the Autograder: `https://c200.luddy.indiana.edu/` and push to GitHub before the due date. **Student Pairs** are provided at the end of this document.

Instructions for submitting to the Autograder

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Given that you already tried points 1-2, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

4. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Some reminders from lecture

Please read through the problems carefully.

- A **constant** function is a function whose output value is the same for every input value. For example: f(x) = 3, irrespective of what we plug in for x, the function f will always output 3.
- Now we know that a **constant** (or fixed) function $f(x)$ returns the same constant value, *e.g.*,

$$f(x, y) = 3 \tag{1}$$

  No matter the inputs, the value is three.

- **Logarithmic Conversion Between Bases** We can convert between $\log_a, \log_k$:

$$\log_b(x) = \frac{\log_k(x)}{\log_k(b)} \tag{2}$$

  Why? Remember that if $\log_b(x) = r$, then $b^r = x$. So, let's first write

$$\log_b(x) = r \tag{3}$$
$$\log_k(x) = s \tag{4}$$
$$\log_k(b) = t \tag{5}$$

  This means

$$b^r = x \tag{6}$$
$$k^s = x \tag{7}$$
$$k^t = b \tag{8}$$

  We see that $b^r = x = k^s$. Since $b = k^t$, we can write:

$$(k^t)^r = k^{rt} = x = k^s \tag{9}$$

  Then

$$\log_k(k^{rt}) = \log_k(x) = \log_k(k^s) \tag{10}$$
$$rt = \log_k(x) = s \tag{11}$$

  Using equations 3,5 for r, t we have

$$\log_b(x) \log_k(b) = \log_k(x) \tag{12}$$
$$\log_b(x) = \frac{\log_k(x)}{\log_k(b)} \tag{13}$$

Problem 1: Functions and math module

All the following functions are drawn from real-world sources. This is an exercise for you to learn how to use a module on your own. From the math module use

- `math.exp(x)` for e^x
- `math.ceil(x)` for $\lceil x \rceil$ rounds x UP to the nearest integer k such that $x \leq k$
- `math.log(x)` for $\log_e(x) = \ln(x)$.

1. According to the Center for Disease Control (CDC) the bacteria *Salmonella* causes about 20K hospitalizations and nearly 400 deaths a year. The formula for how fast this bacteria grows is:

$$N(n_0, m, t) = n_0 e^{mt} \tag{14}$$
$$\tag{15}$$

   where n_0 is the initial number of bacteria, m is the growth rate *e.g.*, 100 per hr, and t is time in hours. Here is how you would calculate the size for an initial colony of 500 that grows at the rate of 100 per hr, for four hours.

$$N(500, 100, 4) = 2.610734844882072 \times 10^{176} \tag{16}$$

2. The number of teeth $N_t(t)$ after t days from incubation for *Alligator mississippiensis* is:

$$N_t(t) = 71.8e^{-8.96e^{-0.0685t}} \tag{17}$$
$$N_t(1000) = \lceil 71.8 \rceil = 72 \tag{18}$$

3. If we want to calculate the work done when an ideal gas expands isothermally (and reversibly) we use for initial and final pressure $P_i = 10\ bar, P_f = 1\ bar$ respectively at $300^o K$. In this problem we are using $\ln$ which is $\log_e$ ('math.log uses base e by default'). Because $\log_e$ is used so often, you'll see it just as often abbreivated as $\ln$.

$$W(P_i, P_f) = RT \ln(P_i/P_f) \tag{19}$$
$$W(10, 1) = \lceil 8.314(300)(\ln 10) \rceil = 5744 \tag{20}$$

   at temperature T (Kelvin) and $R = 8.314\ J/mol \cdot Kelvin$ is the universal gas constant.

4. The Wright Brothers are known for their Flyer and its maiden flight. The formula for lift is:

$$L(V, A, C_\ell) = kV^2 AC_\ell \tag{21}$$
$$L(33.8, 512, 0.515) = \lceil 0.0033(33.8)^2(512)0.515 \rceil = 995 \tag{22}$$

where k is Smeaton's Coefficient ($k = 0.0033$ from their wind tunnel), $V = 33.8\ mph$ is relative velocity over the wing, $A = 512\ ft$ area of wing, and $C_\ell = 0.515$ coefficient of lift. The Flyer weighed $600\,lbs$ and Orville was about $145\,lbs$. We can see that the lift was sufficient since $995 > 745$.

Deliverables for Problem 1

- Complete $N(), N_t(), W(), and\ L()$ functions described above.
- Don't forget to round up using $math.ceil(x)$ as indicated.

Problem 2: More Quadratic

We saw, and also know from basic algebra that for $ax^2 + bx + c = 0$ the roots (values that make the equation zero) are given by:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{23}$$

The expression $b^2 - 4ac$ is called the discriminant. By looking at the discriminant we can deteremine properties of the roots:

- If $b^2 - 4ac > 0$, then both roots are real
- If $b^2 - 4ac = 0$, then both roots are $-b/2a$
- If $b^2 - 4ac < 0$, then both roots are imaginary

There are other properties we can determine from the coeficients. For example, the quadratic as a maximum when $a < 0$, since the graph opens down; the graph opens up when $a > 0$. The vertex of the parabola–the maximum or minimum–is given by:

$$v((a, b, c)) = \frac{-b}{2a} \tag{24}$$

where the coefficients are given as a tuple (a, b, c). By substituting the vertex, you'll find the y-coordinate:

$$f(v((a, b, c))) = f(\frac{-b}{2a}) \tag{25}$$

For example, if $f(x) = -2.6x^2 + 7.6x - 10$, then the maximum point is:

$$(v((a, b, c)), f(v((a, b, c)))) = (\frac{-7.6}{2(-2.6)}, f(\frac{-7.6}{2(-2.6)})) \tag{26}$$

$$\approx (1.46, -4.45) \tag{27}$$

Another example, for $f(x) = x^2 - 10.2x + 26.01$ opens up and has vertex at (5.1,0).

For this function, you'll return a tuple (x,y) where x is a string that says either “down" or “up" and y is a tuple (vertex, f(vertex)) for the quadratic function f. The following code:

```
1 print(q((-2.6,7.6,-10)))
2 print(q((1,-10.2,26.01)))
```

produces:

```
1 ('down', (1.46, -4.45))
2 ('up', (5.1, 0.0))
```

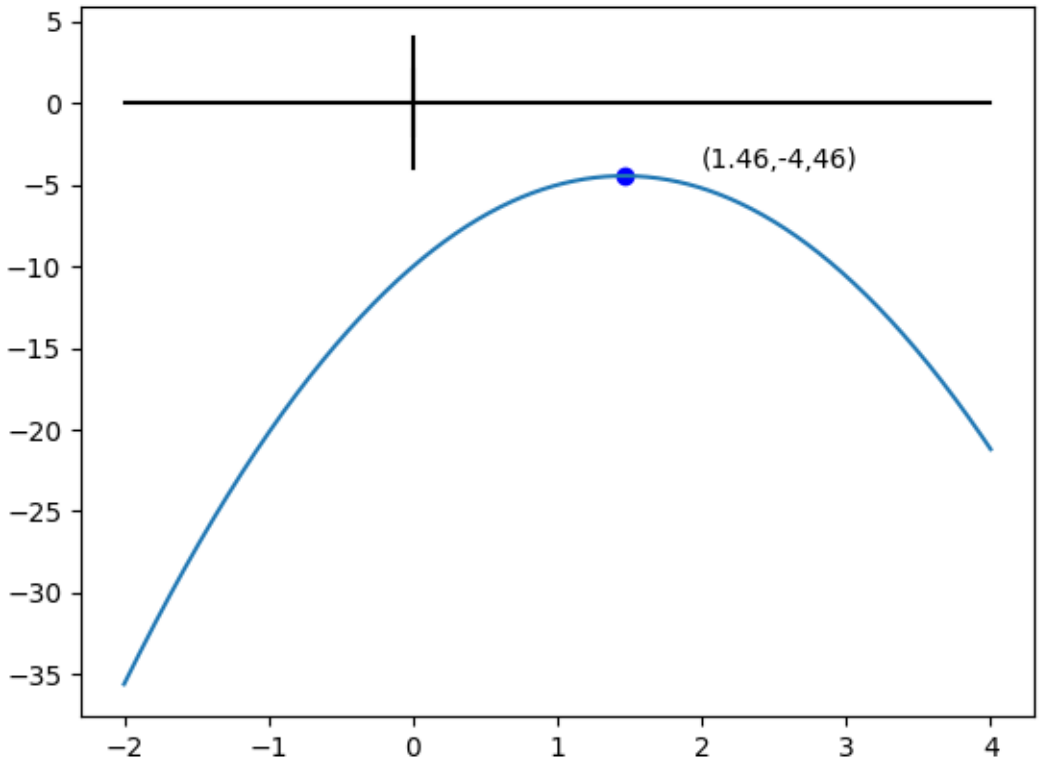


Figure 1: Graphing $f(x) = -2.6x^2 + 7.6x - 10$ observe whethe it opens down (it does) or up and the vertex, y-coordinate pair.

Deliverables for Problem 2

- Complete the functions.
- You are not allowed to use the module `cmath`.
- You will not be asked to examine roots with complex answers.
- When **returning** the vertex,y-coordinate pair, round to two decimal places.

Problem 3: Something Wicked This Way Comes

Search on this phrase “what food isn’t taxed in Indiana”. You’re writing software that allows for customer checkout at a grocery store. A customer will have a have receipt r which is a list of pairs $r = [[i_0, p_0], [i_1, p_1], \ldots, [i_n, p_n]]$ where i is an item and p the cost. You have access to a list of items that are not taxed $no\_tax = [j_0, j_1, \ldots, j_m]$. The tax rate in Indiana is 7%. Write a function amt that takes the receipt and the items that are not taxed and gives the total amount owed. For this function you will have to implement the member function $m(x, lst)$ that returns True if x is a member of lst.

For example, let (receipt) $r = [[1, 1.45], [3, 10.00], [2, 1.45], [5, 2.00]]$, $tax\_rate = \frac{7}{100}$ and $no\_tax = [33, 5, 2]$. Then

$$amt(r, no\_tax) = round(((1.45 + 10.00)1.07 + 1.45 + 2.00), 2) \quad (28)$$

$$= \$15.7 \quad (29)$$

If all items are taxable (for the same receipt), but the tax is 10%, then the total will be $16.39. The following code:

```
1 receipt = [[1,1.45],[3,10.00],[2,1.45],[5,2.00]]
2 tax_rate,no_tax = 7/100, [33,5,2]
3 print(amt(receipt,tax_rate, no_tax))
4 print(amt(receipt,10/100,[]))
```

produces:

```
1 15.7
2 16.39
```

Deliverables for Problem 3

- Complete $m(x, lst)$ function. You must search for x in lst by looping through lst
- You are free to use either an iterable or subscripting
- Complete the $amt()$ function.
- To remind you, Python includes the predicate x in y returns true if x is a member of the compound structure y.

```
1 >>> 1 in [2]
2 False
3 >>> 1 in [2,1]
4 True
5 >>> 1 in 1
6 Traceback (most recent call last):
7   File "<stdin>", line 1, in <module>
8 TypeError: argument of type 'int' is not iterable
```

- Round the output of amt() function to 2 decimal places.

Problem 4: 2D intersecting lines

A line in 2D Euclidean space can be described by $y = mx + b$ (m and b are the slope and intercept respectively). You are provided with a function that takes two points and returns the line as a dictionary:

```
#INPUT two points in 2D
#OUTPUT a dictionary {'m':m, 'b':b} holding slope & intercept
#values are rounded to two decimal places
def make_line(p0,p1):
    x0,y0,x1,y1 = *p0,*p1
    m = round((y1 - y0)/(x1 - x0),2)
    b = round(y0 - (m*x0),2)
    return {'m':m, 'b':b}
```

We'll assume all lines are functions. You'll implement a function that takes two lines described above and return the point of intersection (x, y). Here is the function on two inputs:

```
p0 = (32,32)
p1 = (29,5)
p2 = (15,10)
p3 = (49,25)
p4 = (15,30)
p5 = (50,15)

l0,l1 = make_line(p0,p1),make_line(p2,p3)
print(intersection(l0,l1))
l0 = make_line(p4,p5)
print(intersection(l0,l1))
```

has output:

```
(30.3, 16.73)
(37.99, 20.11)
```

The other two outcomes are either the lines are the same or parallel. In this case return the strings "same lines" or "parallel lines" as shown below:

```
p6,p7,p8 = (0,0),(1,1),(2,2)
p9,p10 = (0,1),(1,2)
print(intersection(make_line(p6,p7),make_line(p7,p8))) # same line
print(intersection(make_line(p6,p7),make_line(p9,p10))) # parallel ↩
    lines
```

has outcome:

```
same line
parallel lines
```

When you run your code and it's correct, the output will be:

```
(30.3, 16.73)
(37.99, 20.11)
same line
parallel lines
```

Deliverables for Problem 4

- Complete the function.
- You cannot use any math module or cmath module function.
- Round the values of slope and intercept to two decimal places.

Problem 5: Means

When analyzing data, we often want to summarize it: make it concise. Each of these functions takes a list of n numbers as nlst. The mean of a list of numbers gives a summary through one number. You're probably aware of the arithmetic mean:

$$\text{arithmetic\_mean}(\text{nlst}) = \frac{(x_0 + x_1 + \cdots + x_{n-1})}{n} \tag{30}$$

For example, the arithmetic mean of [1,2,3] is 2.0.

The geometric mean (usually done with logs) is:

$$\text{geo\_mean}(\text{nlst}) = a^{sum/n} \tag{31}$$

$$sum = \log_a(x_0) + \log_a(x_1) + \cdots + \log_a(x_{n-1}) \tag{32}$$

where $\log_a$ is an arbitrary log to base a. For example, the geometric mean of [2,4,8] is 4.0. Use $\log_{10}$ as default–but it doesn't actually matter. The harmonic mean is:

$$\text{har\_mean}(\text{nlst}) = \frac{n}{1/x_0 + 1/x_1 + \cdots + 1/x_{n-1}} \tag{33}$$

For example, the harmonic mean of [1,2,3] is approximately 1.64.

The root mean square is:

$$\text{RMS\_mean}(\text{nlst}) = \sqrt{\frac{sum}{n}} \tag{34}$$

$$sum = x_0^2 + x_1^2 + \cdots + x_{n-1}^2 \tag{35}$$

For example, the root mean square of [1,3,4,5,7] is approximately 4.47.

All of these functions take a (possibly empty) list of numbers. If there is a list of numbers, then return the mean. If the list is empty, return the string **"Data Error: 0 values"**. If there is a zero in the list of numbers for the harmonic mean, then return the string **"Data Error: 0 in data"**. We give these two errors, because we face division by zero if we don't. The following code:

```
1 print(arithmetic_mean([]))
2 print(arithmetic_mean([1,2,3]))
3 print(geo_mean([]))
4 print(geo_mean([2,4,8]))
5 print(har_mean([]))
6 print(har_mean([1,2,3]))
7 print(har_mean([1,2,0]))
8 print(RMS_mean([1,3,4,5,7]))
```

produces:

```
Data Error: 0 values
2.0
Data Error: 0 values
4.0
Data Error: 0 values
1.6363636363636365
Data Error: 0 in data
4.47213595499958
```

Deliverables for Problem 5

- Complete the functions as specified above.
- In case of empty list or 0 in harmonic mean-return the error string exactly as mentioned in the problem description. Do not add any extra white spaces.

Problem 6: Occurs

In this problem you'll write one function occur_at_least(x,y,lst) that returns True if object x occurs at least y times in lst, False otherwise. We assume lst is either a list or a tuple. Here are a couple of runs:

```
data6 = [[1,4,[1,2,1,2,1,1]], [1,3,[1,2,1,2,1,1]],
         [1,4,(1,2,1,2,1,0)]]

for d in data6:
     print(occur_at_least(*d))
```

has output:

```
True
True
False
```

Deliverables for Problem 6

- Complete the function.
- You **cannot** use any list or tuple functions that count–if you do, you'll receive a zero for this problem.

Problem 7: Looping

We will describe succinctly each kind of function and the type of looping you're required to use. The first function occurs_more(x,y,lst) takes two objects, x,y and a list lst and returns True if x occurs strictly more than y in lst. If the list is empty, then it should return True (since it's not provably false). The second function equal_remove(x,y,lst) returns a list where the number of occurrences of x,y in lst are the equal. If x occurs more, then it is removed from the lst (from left to right) until it occurs the same amount as y. If y occurs, more, then similarly. For example, the following code:

```
1
2       lst = [2,2,3,1,2,1,1,2]
3       print(occurs_more(1,2,lst))
4       print(occurs_more(2,3,lst))
5       print(occurs_more(2,3,[]))
6
7       print(equal_remove(1,2,lst))
8       print(equal_remove(1,3,lst))
9       print(equal_remove(2,3,lst))
10      print(occurs_more(2,3,(equal_remove(2,3,lst))))
```

produces

```
1  False
2  True
3  True
4  [2, 3, 1, 2, 1, 1, 2]
5  [2, 2, 3, 2, 1, 2]
6  [3, 1, 1, 1, 2]
7  False
```

When implementing this function, you're encouraged to use local helper functions. In my code I have this:

```
1  def equal_remove(x,y,lst):
2      tmp = lst
3      def cnt_occurs(x,lst): #returns the count of x in lst
4          pass
5
6      x_c,y_c = cnt_occurs(x,lst),cnt_occurs(y,lst) #find the counts
7
8      def re_k(x,cnt,lst):  #removes the first cnt occurrences of x in ↩
           lst
9          pass
10
```

```
11      if x_c < y_c:
12          tmp = re_k(y,y_c-x_c,tmp)
13      elif x_c > y_c:
14          tmp = re_k(x,x_c - y_c,tmp)
15
16      return tmp
```

This made solving the problem *much* easier. You're free to solve it how you'd like.

Deliverables for Problem 7

- Complete the functions.
- Use either subscripting or iterables.

Problem 8: Geometric Series

A geometric series is the sum of an infinite number of terms that have a constant ratio between successive terms. For example,

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} \tag{36}$$

is apparently a geometric series since:

$$\frac{1/4}{1/2} = \frac{1/8}{1/4} = \frac{1/16}{1/8} = \tfrac{1}{2} \tag{37}$$

Write a function $is\_geo(xlst)$ that takes a list of numbers and returns 1 only if the series is geometric; otherwise 0. If the list has 2 or fewer members, is_geo returns 0. For example,

$$is\_geo([1/2, 1/4, 1/8, 1/16, 1/32]) = 1 \tag{38}$$
$$is\_geo([1, -3, 9, -27]) = 1 \tag{39}$$
$$is\_geo([625, 125, 25]) = 1 \tag{40}$$
$$is\_geo([1/2, 1/4, 1/8, 1/16, 1/31]) = 0 \tag{41}$$
$$is\_geo([1, -3, 9, -26]) = 0 \tag{42}$$
$$is\_geo([625, 125, 24]) = 0 \tag{43}$$
$$is\_geo([1/2, 1/4]) = 0 \tag{44}$$

Deliverables for Problem 8

- Complete the function
- Assume none of the numbers are zero
- If the series has 2 or fewer the function returns 0

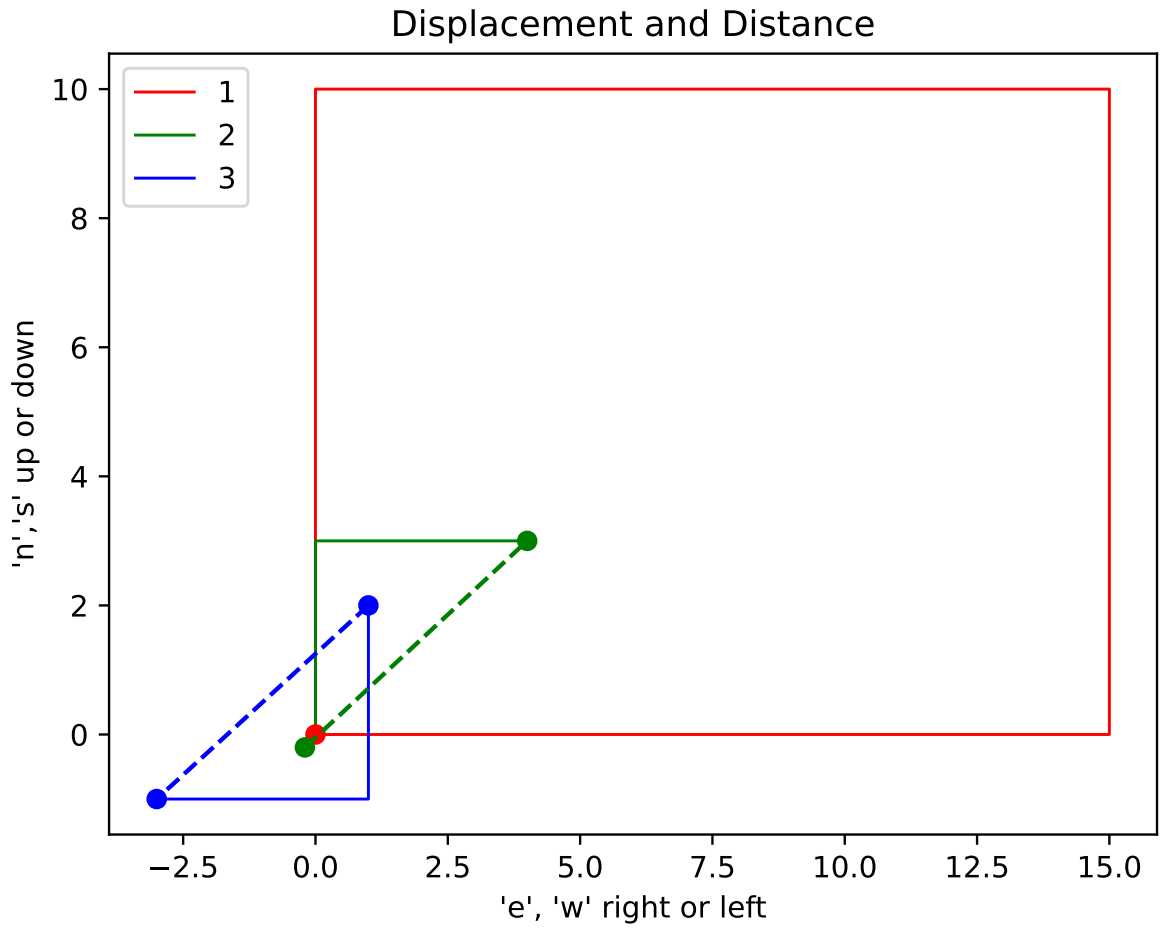


Figure 2: Each color is a run. Since green and red start at the same location, the green is slightly offset so it is visible. The dots show the starting and ending positions. Blue and green have dashed lines, because their starting and ending points are different. The dashed lines both have size 5.

Problem 9: Tracking movement of objects

For this problem, we're tracking movement of objects. This is first step in autonomous vehicle movement. You'll write two functions. The first function determines the distance in 2D space:

$$net\_displacement(p_0, p_1) \quad = \quad [(x_0 - x_1)^2 + (y_0 - y_1)^2]^{1/2}$$

where $p_0 = (x_0, y_0)$ and $p_1 = (x_1, y_1)$. The second function called track(start_pos, movement) takes a starting point (x, y) and a list of movements of size one. The list is the first letter of "east", "west", "north", "south". If the current position is (x, y), then each letter changes the values:

'e'	move right (add one to x)
'w'	move left (subtract one from x)
's'	move down (subtract one from y)
'n'	move up (add one to y)

For example, if you're starting at $(3, 0)$ then ['e','e','w','s','s','s'] results in $(4, -3)$. The total distance travelled is 6 (units). The function returns a tuple: the current location, the distance traveled, and the actual distance between the start and end points. We use net displacement to find how far we are from our starting position. The figure 2 show three calls: red, blue, green. Look at the calls of the function with the graphic (built from Python too!). Here are the three calls:

```
data_m = [[(0,0), list(10*'n' + 15*'e' + 10*'s'+15*'w')],
          [(0,0), list(3*'n' + 4*'e')],
          [(1,2), list(3*'s' + 4*'w')]]

for d in data_m:
    print(track(*d))
```

has output:

```
((0, 0), 50, 0.0)
((4, 3), 7, 5.0)
((-3, -1), 7, 5.0)
```

So, the first run started at (0,0), took 50 steps, and ended-up back where it started. To make encoding more convenient, we take advantage of typecasting from strings to lists. For a string "abc...xyz", list("abc...xyz") returns ['a','b','c',...,'x','y','z'].

Deliverables for Problem 9

- Complete the two functions.
- Round the output of net_displacement to two decimal places.

Problem 10: Distance between two objects

Assume we are tracking two vehicles using the system in problem 9. Write a function final_distance(m0,m1) that takes two tuples (outputs from track) and determines the final distance between the two vehicles. For example,

```
data_m = [[(0,0), list(10*'n' + 15*'e' + 10*'s'+15*'w')],
          [(0,0), list(3*'n' + 4*'e')],
          [(1,2), list(3*'s' + 4*'w')]]

print(final_distance(track(*data_m[1]),track(*(data_m[2]))))
```

gives output:

```
8.06
```

because

$$net\_displacement((4,3),(-3,-1)) \quad = \quad [(4-(-3))^2 + (3-(-1))^2]^{1/2} = [49+16]^{1/2} \approx 8.06$$

As a reminder, the distance between two objects is computed generally as

$$net\_displacement(p_0,p_1) \quad = \quad [(x_0-x_1)^2 + (y_0-y_1)^2]^{1/2}$$

Deliverables for Problem 10

- Complete the function.
- While you are free to fully implement this, problem 9 already has the function(s) you need obviating a lot of coding. You can call the same functions to solve this function and only write new code as needed.

Problem 11:Modeling Politics

You're working for yourself as a data science campaign consultant. You have built a model that predicts, for a political party, the percentage of seats in the House of Representatives h as a function of the percentage of popular vote for that respective party's presidential candidate x as:

$$h(x) = \frac{x^3}{x^3 + (1-x)^3} \tag{45}$$

You realize that you can use this model to ask the complementary question,"If the party wants to win the h percentage seats, what would the presidential candidate's percentage popular vote need to be? " For example, To win about $60\,\%$ of the seats, the candidate must garner about $53.4\,\%$ as we can confirm using the console:

```
>>> x = .534
>>> x**3/(x**3 + (1-x)**3)
0.6007594804866887
```

Your task is two implement a function $h_p(pres)$ that predicts the house seats that are needed for a percentage presidential vote. This code:

```
    pres_per_11 = 60/100
    print(h_p(pres_per_11))
    for pres_per in range(0,100,10):
        print(h_p(pres_per/100))
```

produces:

```
0.534
0.0
0.325
0.386
0.43
0.466
0.5
0.534
0.57
0.614
0.675
```

Deliverables for Problem 11

- This problem uses simple algebra. Work through the arithmetic first.
- Complete the function.
- Round the value to three decimal places.

Problem 12: Go Fund Me

In this problem, you'll be writing code that takes donations for a goal amount that is initially posted. The values are assumed to be dollars. The list contains the individual donations. You'll keep taking donations until the goal is met. The function returns a tuple: the first value is the original goal, the second value is the list of possible donations that are left if the goal is minimally met, and the last value is the amount that is needed to meet the goal. If the goal is not met, the last value returned should be negative. For this problem, using enumerate is very useful. Here are a couple of runs:

```
1 data12 = [[100,[10,15,20,30,29,13,15,40]], [100,[]], [100,[30,4]]]
2
3 for d in data12:
4     print(go_fund_me(*d))
5
6 print(go_fund_me(50, [45,47,78]))
```

has output:

```
1 (100, [13, 15, 40], 4)
2 (100, [], -100)
3 (100, [], -66)
4 (50, [78], 42)
```

The first tuple says the goal of $100 has been met with 10+15+20+30+29 = 104. No more donations are needed. The remainder [13,15,40] are returned to the people and 4 is the amount over. For the second donation, nobody donated so the goal remains unchanged. For the third, only two people donated. The goal that's left is $66.

Deliverables for Problem 12

- Complete the function.

Problem 13: FICO Credit Score

Your access to credit is determined by your FICO (Fair Isaac Corporation) credit score. This is a number [0, 850] that reflects your creditworthiness (a number reflecting what is believed to be the probability you'll not honor your debt obligations–in other words, if you borrow, will you pay it back? This is determined through three ways: model (using AI, statistics, historical data), a human (using tools), or hybrid (both). To calculate the creditworthiness, values of five inputs P,A,L,N,C are weighted (by the following weights) and summed:

- (.35) P (Payment History)
- (.30) A (Amounts Owed)
- (.15) L (Length of Credit History)
- (.10) N (New Credit)
- (.10) C (Credit Mix)

For example, if P = 1, A = 2, L = 3, N = 4 and C = 5 then creditworthiness can be calculate as P * .35 + A * .30 + L * .15 + N * .10 + C * .10.

In this problem we'll write a function loan(cs, lst) which take a credit score and lst as [n,cd], where n is a name and cd is a dictionary that contains a person's five inputs (unweighted). The function loan returns the list of names of people whose creditworthiness is higher than cs, and therefore will be given loan through our bank Republic United National Bank or RUN B. Here is a run of on some applicants:

```
1 data = [['x',{'P':600, 'L':700,'A': 500, 'N': 170, 'C': 250}],
2         ['y',{'P':550, 'L':720,'A': 500, 'N': 230, 'C': 250}],
3         ['b',{'P':560, 'L':710,'A': 500, 'N': 221, 'C': 250}],
4         ['c',{'P':800, 'L':700,'A': 200, 'N': 100, 'C': 150}],
5         ['a',{'P':800, 'L':800,'A': 600, 'N': 250, 'C': 150}],
6         ['z',{'P':800, 'L':800,'A': 500, 'N': 250, 'C': 150}]]
7 print(loan(550,data))
```

producing:

```
1 [['a', 620.0], ['z', 590.0]]
```

Deliverables for Problem 13

- Complete the function

Problem 14: Agatha Christie and Miss Marple

Agatha Christie is among the most widely read mystery novelist. The Miss Marple series is my favorite. In these charming novels, there always is a *murder* to be solved. Let's visit a mystery to be solved with data science and computing.

We have three suspects: Ursala, the famous Malamute animal; Kaiser, her seemingly good-natured German Shephard assistant, Shilah, the sister of Ursala, but kind of a wild child. When our eccentric detective Dr. D arrives on the scene, at 4:00P, the body of a gold fish is found–a large goldfish. The temperature is taken and found to be 81^o F. The dwelling is much colder than the average home–the thermostat says 65^o F. After the suspects are given a cookie and asked to go outside in the backyard, the temperature of the hapless water breather is taken 1 hour later and found to be 77^o F. We find out that the normal body temperature of the gold fish is 85^o F

We have a model that describes how something warmer than the environment eventually becomes cooler.

$$T(t) \quad = \quad T_e + (T_0 - T_e)e^{-kt} \tag{46}$$

where $T(t)$ is the temperature at time t, T_e is the steady temperature of the environment, T_0 is the initial temperature of what's being studied, and k a constant that has to be determined. The data for this problem is:

$$81 \quad = \quad 65 + (85 - 65)e^{-kt} \tag{47}$$

when the fish was discovered. An hour later we found

$$77 \quad = \quad 65 + (85 - 65)e^{-k(t+1)} \tag{48}$$

We have two equations with two unknowns. The suspects couldn't be account for during these times:

Name	Missing
Ursala	3:00P to 4:00P
Kaiser	1:00P to 2:00P
Shilah	2:00P to 2:30P

When we run the numbers, the evidence points to Ursie–she's absconded with her food bowl and my credit cards–be on the lookout.

Deliverables for Problem 14

- Complete the function time that returns how many hours elapsed.
- To do that, you'll need to calculate k. Find k to six decimal places
- The starter code includes the math module. To remind you, math.log() is the computational representative of $\ln()$ (or $\log_e()$).

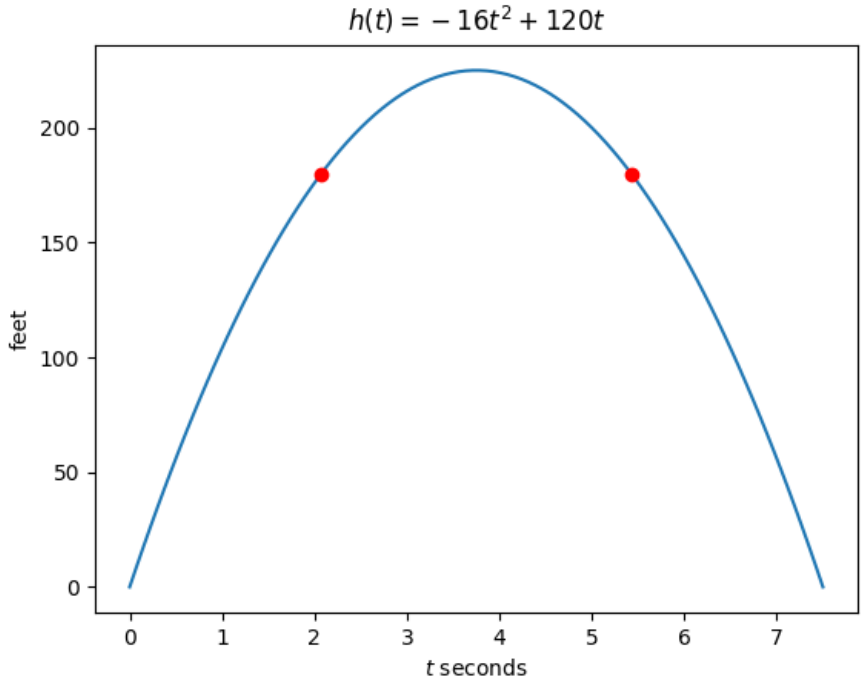
Problem 15: Rocketry

After some experimentation, you have developed a model for a rocket's height (feet) as a function of time t:

$$h(t) = -gt^2 + vt \tag{49}$$

$$h(t) = -16t^2 + 120t \tag{50}$$

In this case $g = -16$ and $v = 120$ (gravity and initial velocity). The rocket's height can be modelled as a parabola:



observing that, except for it's maximal height, each height is reached twice. For this example, we look at times 2.07 sec and 5.43 sec corresponding to 180 ft. Implement a function $rocket(data)$ that takes the height (after t seconds) , gravity, and initial velocity of Eq. 49 and returns the pair of times the rocket is at that height. For example,

```
1 data = (180, -16, 120)
2 print(rocket(data))
```

produces:

```
1 (2.07, 5.43)
```

If you're interested in making this plot (we'll be doing this beginning next week), here's the code:

```
1 import matplotlib.pyplot as plt
2
```

```
t = np.linspace(0,7.5,100)
h = -16*(t**2) + 120*t
plt.plot(t,h)
plt.xlabel(r"$t$ seconds")
plt.ylabel(r"feet")
plt.title(r"$h(t) = -16t^2 + 120t$")
plt.plot([2.07,5.43],[180,180],'o',c='r')
plt.show()
```

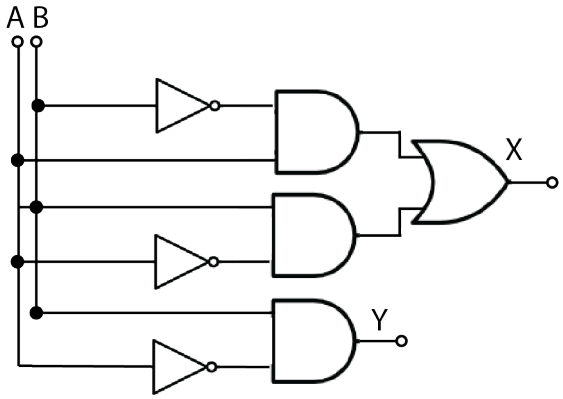
You'll have to install matplotlib. The easiest is using pip. This solution uses arrays. We'll learn about arrays soon!

Deliverables for Problem 15

- Complete the function that returns a pair of times as a tuple (t_0, t_1) where $t_0 \leq t_1$.
- Round to two decimal places.

Problem 16: Circuits and Gates

You're working with a chip making company building digital twins. A digital twin is an *in silico* manifestation of a machine (most large manufacturers are pursuing this). You've been given the schematic that has two inputs (bits) and two outputs. We can observe the logic from the circuit diagram (the black dots mean there's a connection):



A table helps describe the function:

A	B	X	Y
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

The following code:

```
for i in [(0,0),(0,1),(1,0),(1,1)]:
    print(ad(*i))
```

produces:

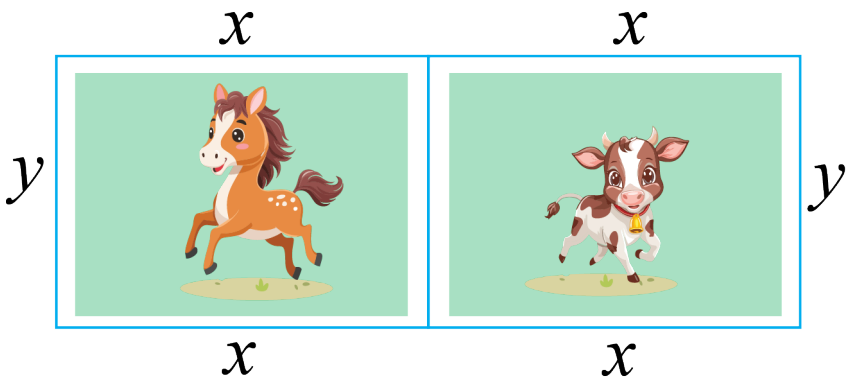
```
(0, 0)
(1, 0)
(1, 0)
(0, 1)
```

Deliverables for Problem 16

- Complete the function that returns a pair (X, Y)
- Make sure to only return integers

Problem 17: Optimization

You're building an enclosure for your miniature horses and cows. The area for both is a rectangle show in the figure below.



You want to use the least amount of fencing. Implement a function `fence(size)` that takes the size of one of the enclosures (they're both the same size, so twice the size isn't needed) and returns a list with the tuple of the dimensions `(x,y)` and the total amount of fencing needed. For this problem, we'll need to use the math function `is_close(a,b)` found at `https://docs.python.org/3/library/math.html`. Specifically, we'll be absolute tolerance. The reason we need this is that the answer doesn't include integers. If you're unfamiliar or rusty with calculus, you can ignore the steps paying attention to (51), (61). Suppose the area is $900\,\text{sq ft}$. We have then:

$$xy = 900 \tag{51}$$
$$fence(x,y) = 3y + 4x \tag{52}$$
$$\tag{53}$$
$$x = \frac{900}{y} \tag{54}$$
$$\tag{55}$$
$$fence(y) = 3y + 4\left(\frac{900}{y}\right) \tag{56}$$
$$\tag{57}$$
$$\frac{\mathrm{d}fence}{\mathrm{d}y} = 3 - 4\left(\frac{900}{y^2}\right) = 0 \tag{58}$$
$$\tag{59}$$
$$4\left(\frac{900}{y^2}\right) = 3 \tag{60}$$
$$4\left(\frac{900}{3}\right) = y^2 \tag{61}$$
$$y^2 = 4(300) \tag{62}$$
$$y = 2(10)\sqrt{3} = 20\sqrt{3} \tag{63}$$
$$x = \frac{900}{20\sqrt{3}} = \frac{900\sqrt{3}}{20 \cdot 3} = \frac{300\sqrt{3}}{20} = 15\sqrt{3} \tag{64}$$

We can use (51) and (61) to display the answers given an area. Here is our function:

```
def analytic_fence(size):
    y = math.sqrt(4*((size)/3))
    x = size/y
    return [(x,y), 3*y + 4*x]
```

We're going to only use integers, however, relying on the range object to iterate over all the possible values (in this case, 1-900). So we need to allow some slight differences in values:

```
 math.isclose((36 * 25),900, abs_tol = 2)
```

returns true, because both values are within two. If we use 1000, however, we have

```
 math.isclose((36 * 25),900, abs_tol = 2)
```

The code:

```
size = 900

print(f"analytic {analytic_fence(size)}")
print(f"non-analytic {fence(size)}")

size = 1000

print(f"analytic {analytic_fence(size)}")
print(f"non-analytic {fence(size)}")
```

produces:

```
analytic [(25.980762113533157, 34.64101615137755), 207.84609690826528]
non-analytic [(25, 36), 208]
analytic [(27.386127875258307, 36.51483716701107), 219.08902300206643]
non-analytic [(27, 37), 219]
```

where

```
>>> 15*math.sqrt(3), 20*math.sqrt(3)
(25.980762113533157, 34.64101615137754)
```

The integer solutions seem quite reasonable.

Deliverables for Problem 17

- Complete the function that returns the list `[(x,y), total_fence]`
- You must use `math.isclose`, since the values won't be exact. We suggest absolute tolerance, but you can experiment with relative though that will be harder. Your results much match ours whichever you use.

Problem 18: Expected Value of a Discrete, Finite Random Variable

For this problem, we assume outcomes are equally likely. We saw in lecture that to model the non-determism of outcomes, we develop the idea of a random variable that transforms the data into a number so we can normalize the set of outcomes associating each with a value $[0, 1]$. The expected value of a random number, denoted E[X] is:

$$\mathsf{E}[\mathsf{X}] \quad = \quad \sum_{x \in \mathsf{X}} x \cdot p(x) \tag{65}$$

That is, the weighted sum of all the values. For example, if we throw two dice and consider the sum, then we have:

```
Original Data
 [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6),
  (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6),
  (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6),
  (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6),
  (5, 1), (5, 2), (5, 3), (5, 4), (5, 5), (5, 6),
  (6, 1), (6, 2), (6, 3), (6, 4), (6, 5), (6, 6)]
Random variable as dictionary key
{2: [(1, 1)], 3: [(1, 2), (2, 1)], 4: [(1, 3), (2, 2), (3, 1)],
 5: [(1, 4), (2, 3), (3, 2), (4, 1)],
 6: [(1, 5), (2, 4), (3, 3), (4, 2), (5, 1)],
 7: [(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)],
 8: [(2, 6), (3, 5), (4, 4), (5, 3), (6, 2)],
 9: [(3, 6), (4, 5), (5, 4), (6, 3)],
10: [(4, 6), (5, 5), (6, 4)], 11: [(5, 6), (6, 5)], 12: [(6, 6)]}
Probabilities
P(X = 2) = 0.027777777777777776
P(X = 3) = 0.05555555555555555
P(X = 4) = 0.08333333333333333
P(X = 5) = 0.1111111111111111
P(X = 6) = 0.1388888888888889
P(X = 7) = 0.16666666666666666
P(X = 8) = 0.1388888888888889
P(X = 9) = 0.1111111111111111
P(X = 10) = 0.08333333333333333
P(X = 11) = 0.05555555555555555
P(X = 12) = 0.027777777777777776
Expected Value of X
6.999999999999999
```

The expected value is:

$$\begin{aligned} \mathsf{E}[\mathsf{X}] \quad &= \quad 2(0.027) + 3(0.055) + 4(0.083) + 5(0.111) + 6(0.138) + \qquad (66) \\ &= \quad 9(0.111) + 10(0.083) + 11(0.055) + 12(0.027) \approx 7 \qquad (67) \end{aligned}$$

From lecture our random variable function is:

```
def X(omega):
    a,b = omega
    return a + b
```

For this problem, assume we throw two dice and take the maximum value:

$$\mathrm{X}[(d_1, d_2)] \quad = \quad \max(d_1, d_2) \tag{68}$$

Your task is to complete the random variable function `X(omega)` and expected value `expected_value(X_)` that takes the dictionary and returns the expected value as described. The following code:

```
#build space of events
Omega = []
for i in range(1,7):
    for j in range(1,7):
        Omega.append((i,j))

#create random variable
random_variable(Omega,X)
```

produces:

```
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6),
 (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6),
 (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6),
 (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6),
 (5, 1), (5, 2), (5, 3), (5, 4), (5, 5), (5, 6),
 (6, 1), (6, 2), (6, 3), (6, 4), (6, 5), (6, 6)]
Random variable as dictionary key
{1: [(1, 1)],
 2: [(1, 2), (2, 1), (2, 2)],
 3: [(1, 3), (2, 3), (3, 1), (3, 2), (3, 3)],
 4: [(1, 4), (2, 4), (3, 4), (4, 1), (4, 2), (4, 3), (4, 4)],
 5: [(1, 5), (2, 5), (3, 5), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), ↩
    (5, 5)],
 6: [(1, 6), (2, 6), (3, 6), (4, 6), (5, 6), (6, 1), (6, 2), (6, 3), ↩
    (6, 4), (6, 5), (6, 6)]}
Probabilities
P(X = 1) = 0.027777777777777776
P(X = 2) = 0.08333333333333333
P(X = 3) = 0.1388888888888889
P(X = 4) = 0.19444444444444445
P(X = 5) = 0.25
P(X = 6) = 0.3055555555555556
```

```
21 Expected Value of X
22 4.472222222222222
```

Deliverables for Problem 18

- Complete the two functions.
- You might use the random variable in lecture and complete the expected value first, since the r.v. was defined in lecture.

Student Pairs

vbhattad@iu.edu, jmsequei@iu.edu
emhjames@iu.edu, tnerkar@iu.edu
ealfalah@iu.edu, seuryu@iu.edu
natkling@iu.edu, ctsvetan@iu.edu
clickc@iu.edu, reshetty@iu.edu
pranchan@iu.edu, jo16@iu.edu
nifredri@iu.edu, rvarrier@iu.edu
kevgross@iu.edu, bgtruong@iu.edu
bowenbm@iu.edu, jjrinald@iu.edu
nikdevra@iu.edu, thebmill@iu.edu
keswar@iu.edu, rasahu@iu.edu
aare@iu.edu, jquansah@iu.edu
colguth@iu.edu, marsh1@iu.edu
bahls@iu.edu, pjminne@iu.edu
augguy@iu.edu, corawang@iu.edu
karbern@iu.edu, sahasrir@iu.edu
moazimi@iu.edu, whitedyl@iu.edu
elehaged@iu.edu, zhangsuo@iu.edu
mthinman@iu.edu, jlaytin@iu.edu
jk314@iu.edu, mjo4@iu.edu
radutta@iu.edu, mszach@iu.edu
twchase@iu.edu, jopantoj@iu.edu
blbinder@iu.edu, rpolu@iu.edu
carthugh@iu.edu, bsati@iu.edu
herncr@iu.edu, cpmathew@iu.edu
ckbarts@iu.edu, mahonm@iu.edu
eynkansa@iu.edu, tunnitha@iu.edu
mahichan@iu.edu, ksabloak@iu.edu
nbriere@iu.edu, mitcal@iu.edu

kbussel@iu.edu, emasseng@iu.edu
sabyrap@iu.edu, auskirvi@iu.edu
goldfusl@iu.edu, lemarc@iu.edu
adj18@iu.edu, dmw6@iu.edu
adodaro@iu.edu, pp22@iu.edu
ebederma@iu.edu, jewach@iu.edu
ethdeatr@iu.edu, olesmith@iu.edu
zahaider@iu.edu, bobbyun@iu.edu
ldeveau@iu.edu, bogawa@iu.edu
btcunnin@iu.edu, kroussel@iu.edu
nwconn@iu.edu, ansjmeht@iu.edu
clarkina@iu.edu, tlichten@iu.edu
abrking@iu.edu, ydpatil@iu.edu
idasilva@iu.edu, klohiya@iu.edu
abondada@iu.edu, fsafianu@iu.edu
nganesh@iu.edu, kyalane@iu.edu
heissler@iu.edu, cjrodenb@iu.edu
yaljabri@iu.edu, omwells@iu.edu
cgcheane@iu.edu, kalovert@iu.edu
alehe@iu.edu, zz67@iu.edu
ldeel@iu.edu, slopezve@iu.edu
jaccryst@iu.edu, drimawi@iu.edu
ygajjar@iu.edu, mt87@iu.edu
adguhan@iu.edu, ozsachs@iu.edu
evbutts@iu.edu, nieywill@iu.edu
meiyhe@iu.edu, mjseymou@iu.edu
okirchen@iu.edu, samyadav@iu.edu
jagoodal@iu.edu, braxluns@iu.edu
ttfender@iu.edu, shamonei@iu.edu
jbriar@iu.edu, rheashah@iu.edu
damaatki@iu.edu, isvoor@iu.edu
agotanco@iu.edu, cpmann@iu.edu
ehchoo@iu.edu, lvumpa@iu.edu
bgelard@iu.edu, clushell@iu.edu
shkani@iu.edu, roldenbu@iu.edu
gkreutes@iu.edu, kenmoses@iu.edu
eberens@iu.edu, cnshimi@iu.edu
benchart@iu.edu, cmolaski@iu.edu
bbeer@iu.edu, rvasude@iu.edu
rbernfel@iu.edu, fasrasoo@iu.edu
coubenne@iu.edu, kr15@iu.edu

ag46@iu.edu, ayorgen@iu.edu
joglickm@iu.edu, hmahapat@iu.edu
eabilius@iu.edu, esspatel@iu.edu
byersjac@iu.edu, kalerobi@iu.edu
cdasari@iu.edu, jrwaren@iu.edu
jaeblank@iu.edu, sumuth@iu.edu
mahchaud@iu.edu, roberjuv@iu.edu
lafly@iu.edu, dyllopez@iu.edu
andedaws@iu.edu, taylbryo@iu.edu
cedrchen@iu.edu, audscha@iu.edu
kadlakha@iu.edu, everma@iu.edu
bcarpine@iu.edu, anupand@iu.edu
mabana@iu.edu, kokulski@iu.edu
concraft@iu.edu, nealsing@iu.edu
hdjour@iu.edu, jwubelho@iu.edu
iblay@iu.edu, jakpatel@iu.edu
mdevara@iu.edu, jnimmala@iu.edu
banksda@iu.edu, cmellgre@iu.edu
zalsamri@iu.edu, ainlukas@iu.edu
lddial@iu.edu, jasreese@iu.edu
bthenson@iu.edu, pmiesch@iu.edu
vicbarre@iu.edu, jakepars@iu.edu
cjo3@iu.edu, nmeely@iu.edu
laigrant@iu.edu, jvmenach@iu.edu
btbasing@iu.edu, gsskippe@iu.edu
jahjacks@iu.edu, athason@iu.edu
enkarl@iu.edu, rnalluri@iu.edu
nfarzane@iu.edu, kpazdur@iu.edu
mgbekele@iu.edu, conaitis@iu.edu
tbashee@iu.edu, vnathany@iu.edu
alurs@iu.edu, smitfi@iu.edu
vkabra@iu.edu, aidsteph@iu.edu
ahsans@iu.edu, kiloscot@iu.edu
ethhanna@iu.edu, evanmayo@iu.edu
hayburge@iu.edu, mmalviya@iu.edu
mitja@iu.edu, vpotluri@iu.edu
jigarnes@iu.edu, ncw1@iu.edu
pikhatri@iu.edu, negia@iu.edu
talfares@iu.edu, aluckhau@iu.edu
jergalan@iu.edu, muksingh@iu.edu
bdgonzal@iu.edu, prmundra@iu.edu

owencape@iu.edu, sholove@iu.edu
jamcham@iu.edu, hsh1@iu.edu
adekolaw@iu.edu, anryoun@iu.edu
etferba@iu.edu, jiajtang@iu.edu
maboncos@iu.edu, olsoncs@iu.edu
dayinla@iu.edu, aidpride@iu.edu
dajuhnke@iu.edu, aramanu@iu.edu
iyersan@iu.edu, vpremku@iu.edu
svdatla@iu.edu, sstickl@iu.edu
skapure@iu.edu, msharsh@iu.edu
langarbe@iu.edu, adimaran@iu.edu
rogoenka@iu.edu, lydiwang@iu.edu
doandre@iu.edu, tersacks@iu.edu
edeitchl@iu.edu, jawegman@iu.edu
lajith@iu.edu, mmaplet@iu.edu
felkhatt@iu.edu, neenpate@iu.edu
nimjain@iu.edu, wl52@iu.edu
dgiffor@iu.edu, robreese@iu.edu
blstcoop@iu.edu, prschulz@iu.edu
elscheng@iu.edu, lisalaza@iu.edu
florech@iu.edu, vivitran@iu.edu
jb108@iu.edu, ssp4@iu.edu
mbenites@iu.edu, parishar@iu.edu
adatwani@iu.edu, amarynow@iu.edu
salvchri@iu.edu, shwanara@iu.edu
dakomi@iu.edu, isshetty@iu.edu
tborder@iu.edu, aposton@iu.edu
jackchap@iu.edu, rosanghv@iu.edu
ahmealsh@iu.edu, ew38@iu.edu
brjduck@iu.edu, isundara@iu.edu
drmiguth@iu.edu, sandro@iu.edu
rpkeatin@iu.edu, ssanjee@iu.edu
shkami@iu.edu, alejmora@iu.edu, mzuhair@iu.edu
sanjais@iu.edu, nshnoble@iu.edu
michdool@iu.edu, anangla@iu.edu
ssdatla@iu.edu, gsackie@iu.edu
mjfarrin@iu.edu, nmenne@iu.edu
addfishe@iu.edu, kw125@iu.edu
jdkakare@iu.edu, manpate@iu.edu
rkoranne@iu.edu, dmonroyh@iu.edu
sydforem@iu.edu, adivaidy@iu.edu

nantinao@iu.edu, adenyu@iu.edu
adhimat@iu.edu, mloko@iu.edu
akeshav@iu.edu, jhl14@iu.edu
zkabacin@iu.edu, aishravi@iu.edu
ajafowl@iu.edu, mawlorch@iu.edu
asgurram@iu.edu, joevu@iu.edu
dqchoe@iu.edu, cammlanc@iu.edu
aabdulw@iu.edu, kzerbino@iu.edu
ljhorman@iu.edu, atsarver@iu.edu
ejaini@iu.edu, mtay@iu.edu
yc127@iu.edu, aamuse@iu.edu
garmenda@iu.edu, antzamor@iu.edu
ndodoh@iu.edu, marcmarq@iu.edu
bedani@iu.edu, ijrussel@iu.edu
armstan@iu.edu, camoverm@iu.edu
coobratt@iu.edu, swaschow@iu.edu
ag38@iu.edu, wschrama@iu.edu
bronburk@iu.edu, vpottete@iu.edu
sergonza@iu.edu, davepais@iu.edu
agrief@iu.edu, rysalina@iu.edu
reclay@iu.edu, sasubb@iu.edu
nitiarun@iu.edu, isapittm@iu.edu
ramgowda@iu.edu, smungee@iu.edu
gibsontn@iu.edu, angzamor@iu.edu
jkuk@iu.edu, mkreddy@iu.edu
srkagema@iu.edu, tkthang@iu.edu
grhamlin@iu.edu, lukthiel@iu.edu
mkulekci@iu.edu, soyedele@iu.edu
rkoston@iu.edu, andyni@iu.edu
aarepal@iu.edu, stewrach@iu.edu
tbrose@iu.edu, ndmyer@iu.edu
xmdrew@iu.edu, jw363@iu.edu
mrb9@iu.edu, antonegr@iu.edu
nafike@iu.edu, froy@iu.edu
fcohan@iu.edu, risvphil@iu.edu
dbarbari@iu.edu, sxanders@iu.edu
chfurlow@iu.edu, timolaws@iu.edu
boydmad@iu.edu, dematt@iu.edu
agahlau@iu.edu, wkpierce@iu.edu
bboyapat@iu.edu, jacdstev@iu.edu
dh43@iu.edu, cnorcutt@iu.edu

sdondeti@iu.edu, elwesley@iu.edu
ajkoutsa@iu.edu, cardnorr@iu.edu
rycostin@iu.edu, matyun@iu.edu
cmfalk@iu.edu, navann@iu.edu
alneyadk@iu.edu, nguyen17@iu.edu
efoltyn@iu.edu, mmmatias@iu.edu
akoduru@iu.edu, alzemlya@iu.edu
owflynn@iu.edu, irodeman@iu.edu
crigonza@iu.edu, hailreid@iu.edu
mfbah@iu.edu, anrajput@iu.edu
mugabr@iu.edu, dommonte@iu.edu
rugaddam@iu.edu, jskains@iu.edu
rdotsu@iu.edu, missavin@iu.edu
skalluru@iu.edu, tanilitt@iu.edu
bskendal@iu.edu, bawsung@iu.edu
xbchrist@iu.edu, aralover@iu.edu
keswa@iu.edu, hshyama@iu.edu
jkittur@iu.edu, cs155@iu.edu
dkembert@iu.edu, youngct@iu.edu
chjflor@iu.edu, maymat@iu.edu
tycarmo@iu.edu, rvenigal@iu.edu
ojallow@iu.edu, ibshaw@iu.edu
jjithesh@iu.edu, martibr@iu.edu
dheer@iu.edu, djtheodo@iu.edu
skyangel@iu.edu, mclindbe@iu.edu
egilmour@iu.edu, liaobrie@iu.edu
drussow@iu.edu, abbywebe@iu.edu
rgyasini@iu.edu, kevwong@iu.edu
mgoenka@iu.edu, jl367@iu.edu
nicdelg@iu.edu, ishyadav@iu.edu
heoji@iu.edu, dajshaw@iu.edu
adagar@iu.edu, natkwon@iu.edu
kkunshet@iu.edu, smitaa@iu.edu
bllfost@iu.edu, gl25@iu.edu
ghargro@iu.edu, westje@iu.edu
smhamrah@iu.edu, ozopich@iu.edu
pchitwoo@iu.edu, prnemani@iu.edu
evarcarm@iu.edu, zs15@iu.edu
am280@iu.edu, cjlomax@iu.edu
mailic@iu.edu, jacloise@iu.edu
jejea@iu.edu, cmekat@iu.edu

jbosio@iu.edu, sunathan@iu.edu